

Vector Search & Semantic Indexing: A Deep Dive

Technology walkthrough for the LIS 467 Node.js demo

LIS 467 — Client/Server & Full-Stack Architectures

October 2026

Vector Search & Semantic Indexing: A Deep Dive

This document works through every technology and every file in the vector-search-demo project, explaining *what* each piece does, *why* it was built that way, and linking to primary documentation for further study.

1. The Problem This Project Solves

Relational databases and document stores (SQL, MongoDB) retrieve data through **exact or pattern-based matching**: a WHERE title = 'X', a LIKE '%word%', a Mongo \$regex. This works well when the user's words match the data's words. It breaks down the moment a user searches with *different words that mean the same thing* — “organizing books” vs. “cataloging and classification.”

Semantic search solves this by representing text as points in a high-dimensional vector space (an **embedding**), where distance between points corresponds to similarity of *meaning* rather than similarity of *characters*. This is the retrieval mechanism underpinning modern **Retrieval-Augmented Generation (RAG)** systems, recommendation engines, and semantic document search.

Reference reading: - Pinecone, *What is a Vector Database?* — <https://www.pinecone.io/learn/vector-database/> - Google Cloud, *Vector search overview* — <https://cloud.google.com/vertex-ai/docs/vector-search/overview> - MongoDB, *What Is Semantic Search?* — <https://www.mongodb.com/resources/basics/semantic-search>

2. Project Architecture at a Glance

```
documents.json --(embed)--> index.json --(load at boot)--> Express server
query string --(embed)-----> cosine similarity
                                   |
                                   | ranked JSON results
                                   |
                                   | browser UI (fetch)
```

Two phases:

1. **Offline indexing** (scripts/buildIndex.js) — run once (or whenever documents change) to convert text into vectors and persist them.
2. **Online querying** (src/server.js) — runs per request, embeds the incoming query and compares it against the pre-computed vectors.

This separation (index once, query many times) mirrors how production vector databases work: embedding generation is comparatively expensive, so it is batched offline wherever possible, while similarity search at query time must be fast.

3. Embeddings: Turning Text into Vectors

3.1 What is an embedding?

An embedding is a fixed-length array of floating-point numbers (a vector) produced by a neural network, such that semantically similar inputs produce vectors that are *close together* under some distance metric (usually cosine similarity or Euclidean distance). The model used here, **Xenova/all-MiniLM-L6-v2**, outputs **384-dimensional** vectors. It is a distilled version of sentence-transformers/all-MiniLM-L6-v2, trained specifically to produce good *sentence-level* embeddings (as opposed to word-level embeddings like classic word2vec).

Reference reading: - Hugging Face model card — <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2> - Sentence-Transformers documentation — <https://www.sbert.net/> - Xenova's ONNX-converted version used in this project — <https://huggingface.co/Xenova/all-MiniLM-L6-v2>

3.2 @xenova/transformers (Transformers.js)

This is a JavaScript port of Hugging Face's Python transformers library, compiled to run **entirely inside Node.js (or even the browser)** using [ONNX Runtime](#) under the hood — no Python process, no external API call, no API key. The library downloads the model weights (in ONNX format) from the Hugging Face Hub on first use and caches them locally.

Reference reading: - Transformers.js documentation — <https://huggingface.co/docs/transformers.js/index> - Transformers.js GitHub repo — <https://github.com/xenova/transformers.js> - ONNX Runtime overview — <https://onnxruntime.ai/docs/>

3.3 Code walkthrough: src/embeddings.js

```
import { pipeline } from "@xenova/transformers";

let embedderPromise = null;

function getEmbedder() {
  if (!embedderPromise) {
    embedderPromise = pipeline("feature-extraction", "Xenova/all-MiniLM-L6-v2");
  }
  return embedderPromise;
}

export async function embedText(text) {
  const embedder = await getEmbedder();
  const output = await embedder(text, { pooling: "mean", normalize: true });
  return Array.from(output.data);
}
```

- **pipeline("feature-extraction", modelName)** — Transformers.js's high-level API. "feature-extraction" is the *task* name for "give me a vector representation of this text" (as opposed to "text-classification", "translation", etc.). This is the same pipeline() abstraction documented at <https://huggingface.co/docs/transformers.js/pipelines>.
- **Lazy singleton pattern** (embedderPromise) — loading the model is slow (downloading/parsing weights), so it is done once and cached in module scope, not on every call. Because pipeline(...) returns a Promise, storing the *promise itself* (not just the resolved value) means concurrent calls to getEmbedder()

before the first load finishes will all await the same in-flight load rather than triggering duplicate loads — a common async singleton pattern in Node.js.

- **pooling: "mean"** — the underlying transformer model actually produces one vector *per input token*, not one vector for the whole sentence. Mean pooling averages all token vectors together into a single sentence vector. This is the standard approach used by Sentence-Transformers models (see the SBERT documentation above).
- **normalize: true** — scales the resulting vector to unit length (L2-normalization, i.e. $\|v\| = 1$). This is what makes the cosine-similarity shortcut in `vectorStore.js` valid (explained in section 4).
- **Array.from(output.data)** — the raw output is a typed array (`Float32Array`) wrapped in a `Transformers.js` Tensor object; converting it to a plain JS array makes it trivial to `JSON.stringify` for storage in `index.json`.

4. Cosine Similarity: The Math Behind “Semantic Closeness”

4.1 The formula

For two vectors A and B , cosine similarity is defined as:

$$\text{cosine_similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

It measures the cosine of the angle between the two vectors, ranging from -1 (opposite meaning) to 1 (identical direction/meaning), with 0 meaning “unrelated.” Unlike Euclidean distance, cosine similarity ignores vector *magnitude* and only cares about *direction* — which is desirable for text embeddings, where magnitude can vary with text length but direction encodes meaning.

Reference reading: - Wikipedia, *Cosine similarity* — https://en.wikipedia.org/wiki/Cosine_similarity - Pinecone, *Vector Similarity Explained* — <https://www.pinecone.io/learn/vector-similarity/>

4.2 Why the code just does a dot product

```
export function cosineSimilarity(a, b) {
  let dot = 0;
  for (let i = 0; i < a.length; i++) {
    dot += a[i] * b[i];
  }
  return dot;
}
```

Because `embedText()` already L2-normalizes every vector (`normalize: true`), $\|A\| = \|B\| = 1$ for every vector produced in this app. Substituting that into the formula above collapses the denominator to 1:

$$\text{cosine_similarity}(A, B) = \frac{A \cdot B}{1 \times 1} = A \cdot B$$

So the plain dot product *is* the cosine similarity here — this is a standard optimization in production vector databases as well (pgvector’s `vector_cosine_ops` operator class and FAISS’s inner-product indexes both rely on this same pre-normalization trick for speed).

4.3 `semanticSearch()` and `keywordSearch()`

```
export function semanticSearch(queryVector, indexedDocs, topK = 5) {
  return indexedDocs
    .map((doc) => ({ ...doc fields..., score: cosineSimilarity(queryVector, doc.embedding)
    }))
    .sort((a, b) => b.score - a.score)
    .slice(0, topK);
}
```

This is a **brute-force linear scan**: every stored document's vector is compared against the query vector, one at a time, in $O(n)$ time. For 16 documents this is instantaneous. Production systems with millions of vectors instead use **Approximate Nearest Neighbor (ANN)** indexes — most commonly **HNSW** (Hierarchical Navigable Small World graphs) — to find near-matches in roughly logarithmic time without scanning every row. Section 7 maps this scan directly onto what pgvector/MongoDB Atlas do instead.

keywordSearch() is included purely as a contrasting baseline: it lowercases the query and the document text, splits the query into terms, and counts how many terms appear anywhere in the document as a substring. It has no concept of meaning, synonyms, or word order — which is exactly the limitation the demo is designed to expose.

Reference reading: - Malkov & Yashunin, *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs* (the HNSW paper) — <https://arxiv.org/abs/1603.09320> - pgvector README (includes HNSW vs. IVFFlat index types) — <https://github.com/pgvector/pgvector>

5. The Indexing Script: scripts/buildIndex.js

```
const docs = JSON.parse(await readFile(DOCS_PATH, "utf-8"));
const indexed = [];
for (const doc of docs) {
  const embedding = await embedText(`${doc.title}. ${doc.text}`);
  indexed.push({ ...doc, embedding });
}
await writeFile(INDEX_PATH, JSON.stringify(indexed, null, 2));
```

Key design points:

- **Title + text concatenation** — embedding `\${doc.title}. \${doc.text}` rather than just the body text lets the title's keywords contribute to the semantic signature of the document, which matters a lot for short documents where the title carries most of the topical meaning.
- **Sequential await in a for loop** (not Promise.all) — this intentionally processes documents one at a time rather than in parallel. The embedding model runs synchronously on the CPU inside the same Node process; running many embeddings concurrently would contend for the same CPU resources without actually speeding things up, and would make the progress log (embedded doc-1...) harder to follow in a demo.
- **Separation from documents.json** — index.json is a *derived artifact* (hence it's gitignored — see .gitignore). documents.json is the source of truth; index.json can always be regenerated by re-running npm run build-index. This mirrors how a real pipeline treats a search index as a rebuildable cache, not as authoritative data.

Node.js APIs used here, for reference: - node:fs/promises (readFile/writeFile) — <https://nodejs.org/api/fs.html#promises-api> - node:url fileURLToPath and ESM import.meta.url — needed because ES modules don't have the CommonJS __dirname global; this is the documented workaround — <https://nodejs.org/api/esm.html#importmetaurl>

6. The Express Server: `src/server.js`

```
import express from "express";
...
const app = express();
app.use(express.json());
app.use(express.static(path.join(__dirname, "..", "public")));

app.get("/api/search", async (req, res) => {
  const query = (req.query.q || "").trim();
  if (!query) return res.status(400).json({ error: "Missing query parameter 'q'" });

  const topK = Number(req.query.k) || 5;
  const queryVector = await embedText(query);

  res.json({
    query,
    semantic: semanticSearch(queryVector, indexedDocs, topK),
    keyword: keywordSearch(query, indexedDocs, topK)
  });
});
```

- **`express.static(...)`** serves the `public/` folder directly, so `index.html`, `style.css`, and `app.js` are available with zero additional routing code. Documented at <https://expressjs.com/en/starter/static-files.html>.
- **GET `/api/search?q=...&k=...`** — a deliberately simple REST-style query endpoint. Using **query parameters** (`req.query`) rather than a POST body is idiomatic for a read-only, idempotent, cacheable search operation (the same reasoning search engines and most “list/search” REST APIs follow). Express query-string parsing is documented at <https://expressjs.com/en/api.html#req.query>.
- **Input validation** — the handler explicitly checks for an empty query and returns `400 Bad Request` rather than silently embedding an empty string. This is a basic but important input-boundary check (see Security Notes, section 9).
- **`indexedDocs` loaded once at boot** — `start()` reads `index.json` a single time into memory before calling `app.listen(...)`; every request afterward reuses this in-memory array rather than re-reading the file from disk, which is why the brute-force scan in section 4.3 stays fast.
- **Fail-fast startup** — if `index.json` doesn’t exist yet (user forgot to run `npm run build-index`), the server logs a clear instruction and calls `process.exit(1)` rather than starting in a broken state. This is a deliberate “fail loud at boot, not silently at request time” choice.

Reference reading: - Express.js official guide — <https://expressjs.com/en/guide/routing.html> - REST API design basics (query params for search/filter) — <https://restfulapi.net/>

7. The Frontend: `public/index.html`, `style.css`, `app.js`

This is intentionally a **plain, dependency-free** frontend — no React, no build step — to keep the technology surface area small for a classroom demo focused on the backend/retrieval concepts.

```
const res = await fetch(`/api/search?q=${encodeURIComponent(query)}&k=5`);
const data = await res.json();
renderResults(semanticList, data.semantic);
renderResults(keywordList, data.keyword);
```

- **`fetch()`** — the standard browser API for making HTTP requests; documented at https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API.

- **encodeURIComponent(query)** — escapes special characters (spaces, &, ?, etc.) so the query string is well-formed; this is the standard, MDN-documented way to safely embed user input into a URL — https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/encodeURIComponent.
- **Side-by-side rendering** — the UI deliberately shows `data.semantic` and `data.keyword` in two parallel `<ol>` lists so the contrast between the two retrieval strategies is visually immediate, reinforcing the demo's central teaching point.

8. Dataset Design: data/documents.json

The 16 sample documents were deliberately chosen to span **five topic clusters** (Computer Science, Library Science, History, Cooking, Gardening) with **multiple documents per cluster**. This matters pedagogically: a single semantic hit could be a coincidence, but when a query like *“organizing books so people can find them”* consistently ranks **all** the Library Science documents (cataloging, metadata standards, information retrieval) above unrelated clusters, it demonstrates that the model has captured a genuine topical/semantic structure — not just noise.

9. Security Notes (OWASP-Informed)

A few things worth calling out, since this is also a teaching moment on safe coding practice:

- **No HTML templating of user input in results** — `app.js` builds result `<li>` elements with `innerHTML` using `document` fields (`title/text/ category`), which come from the trusted local `documents.json`, not from the user's query. The **query string itself is never reflected back into the DOM**, which avoids a reflected XSS vector. If this app were extended to allow user-submitted documents, those fields would need HTML-escaping before insertion via `innerHTML` (or safer, use `textContent`).
- **Input validation at the API boundary** — `server.js` rejects empty queries with 400 rather than passing arbitrary/empty input straight into the embedding model.
- **No secrets/API keys** — because embeddings are generated locally via `Transformers.js`, there is no API key to leak or rate-limit to manage, eliminating an entire class of credential-exposure risk common in cloud-embedding-API-based RAG apps.
- **Known dependency advisories** — `npm audit` flags vulnerabilities in `onnxruntime-web/sharp`, transitive dependencies of `@xenova/transformers` used for local model execution. They are not reachable through this app's HTTP surface (no user-controlled file/image input reaches those libraries), but should be reviewed before any production reuse. See OWASP's *Using Components with Known Vulnerabilities* guidance — https://owasp.org/www-project-top-ten/2017/A9_2017-Using_Components_with_Known_Vulnerabilities.

10. Mapping This Demo to Production Systems

Demo component	Production equivalent	Docs
data/index.json flat array + linear scan	PostgreSQL pgvector column + HNSW index	https://github.com/pgvector/pgvector
Same linear scan	MongoDB Atlas Vector Search (\$vectorSearch aggregation stage)	https://www.mongodb.com/docs/atlas/atlas-vector-search/
@xenova/transformers local model	OpenAI Embeddings API (text-embedding-3-	https://platform.openai.com/docs/guides/embeddings

Demo component	Production equivalent	Docs
	small)	
GET /api/search returning ranked docs	Retrieval step of a RAG pipeline (results fed into an LLM prompt)	https://www.pinecone.io/learn/retrieval-augmented-generation/
Manual cosine similarity loop	FAISS / Pinecone / Milvus ANN search	https://github.com/facebookresearch/faiss , https://milvus.io/docs

11. Summary

This project demonstrates, end to end, the minimal viable version of every concept that underlies production semantic search systems: generating embeddings, storing them alongside source documents, measuring similarity with cosine distance, and exposing the result through a conventional Express/REST API consumed by a plain HTML/JS frontend. Every production vector search system — whether built on pgvector, MongoDB Atlas, or a dedicated vector database — is an optimized, horizontally-scaled version of exactly this pipeline.